Laboratório Funcional · Aula 12

Heitor Henrique Scramim de Freitas · RA 24190382-2 · Turma A
Gerado em 16/10/2026, 20:31:12 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

agora(): o resultado não depende só dos parâmetros (ela nem tem parâmetros). Ela lê o relógio do sistema, que muda o tempo todo, então duas chamadas iguais vão devolver valores diferentes. Falha a condição 1.
registra(msg): tem efeito colateral, pois escreve na tela. Se eu trocar a chamada pelo valor que ela devolve (None), a mensagem some, então não passa no teste da transparência referencial. Falha a condição 2.

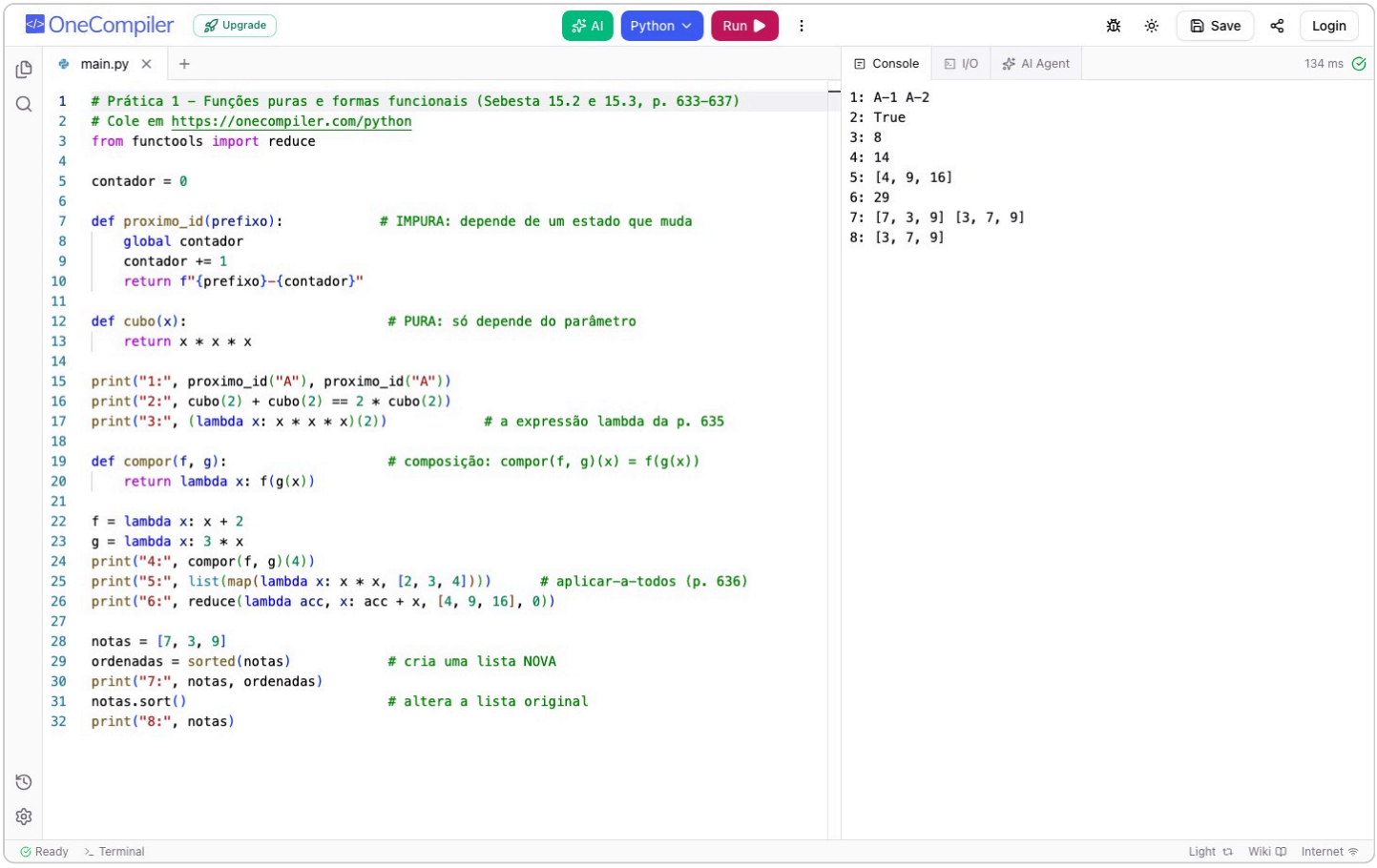
Prática 1: Funções puras e formas funcionais

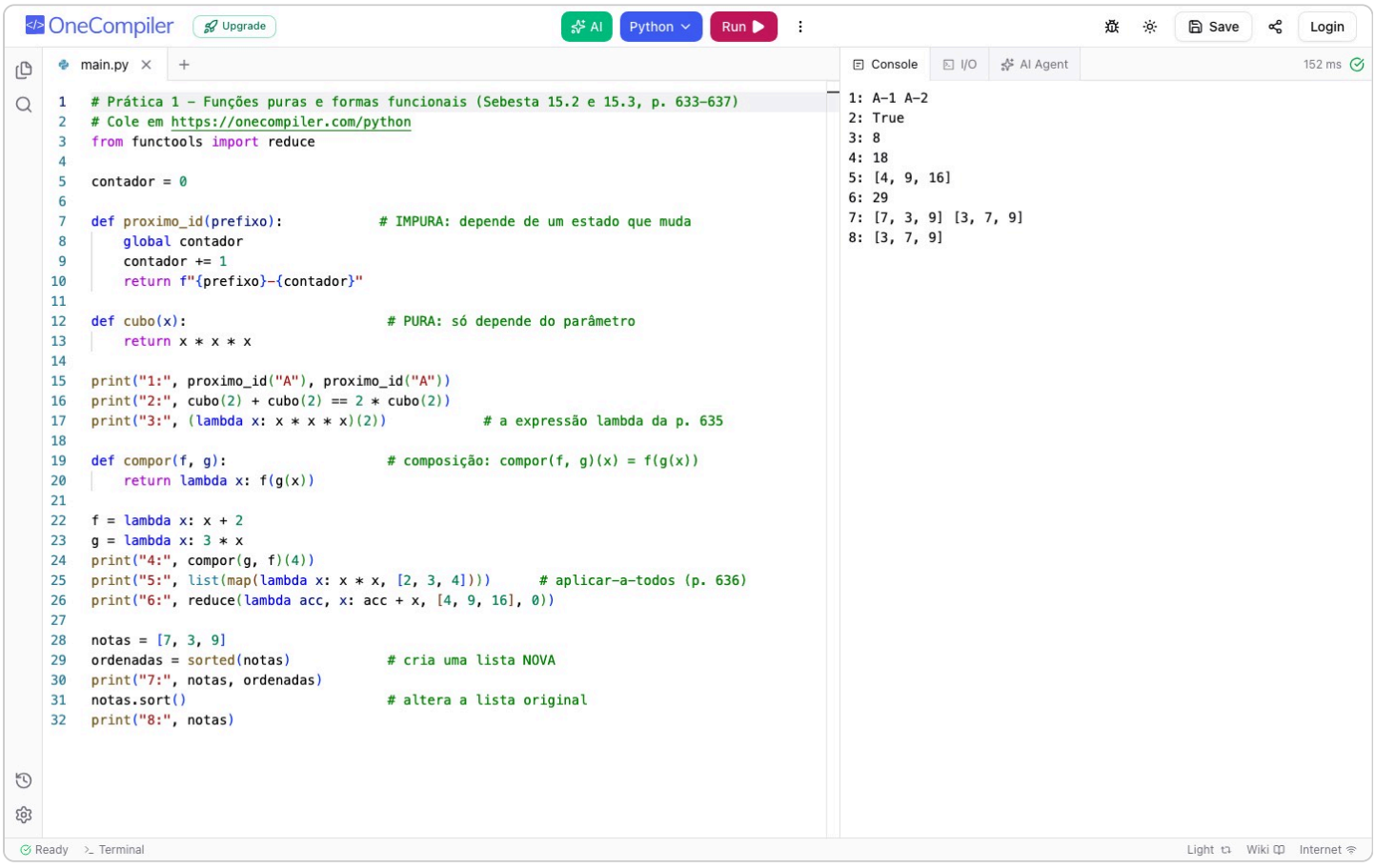
Previsão: O que as linhas 1, 4 e 7 vão imprimir?

1: A-1 A-2
4: 14
7: [7, 3, 9] [3, 7, 9]

Saídas (original e com compor(g, f))

não respondido





1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

Porque proximo_id depende da variável global contador, que ela mesma altera a cada chamada (efeito colateral). O resultado depende do histórico de chamadas e não só do parâmetro: a 1ª chamada dá A-1, a 2ª dá A-2. Isso dificulta

os testes porque a função não pode ser testada isoladamente: o resultado esperado muda conforme a ordem e a quantidade de chamadas anteriores, e é preciso reiniciar o estado global antes de cada teste. Não há transparência referencial.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

compor(f, g)(4) = f(g(4)) = f(3*4) = f(12) = 12 + 2 = 14.
compor(g, f)(4) = g(f(4)) = g(4 + 2) = g(6) = 3*6 = 18.
A composição não é comutativa, a função da direita é aplicada primeiro. Trocando a ordem, muda qual operação acontece antes (multiplicar->somar, ou somar->multiplicar).

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) cria e devolve uma nova lista ordenada e não mexe na original.
notas.sort() ordena a própria lista no lugar (altera o original) e devolve None. O sorted segue o estilo funcional, porque não tem efeito colateral, só recebe um valor e devolve outro, sem modificar o parâmetro.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

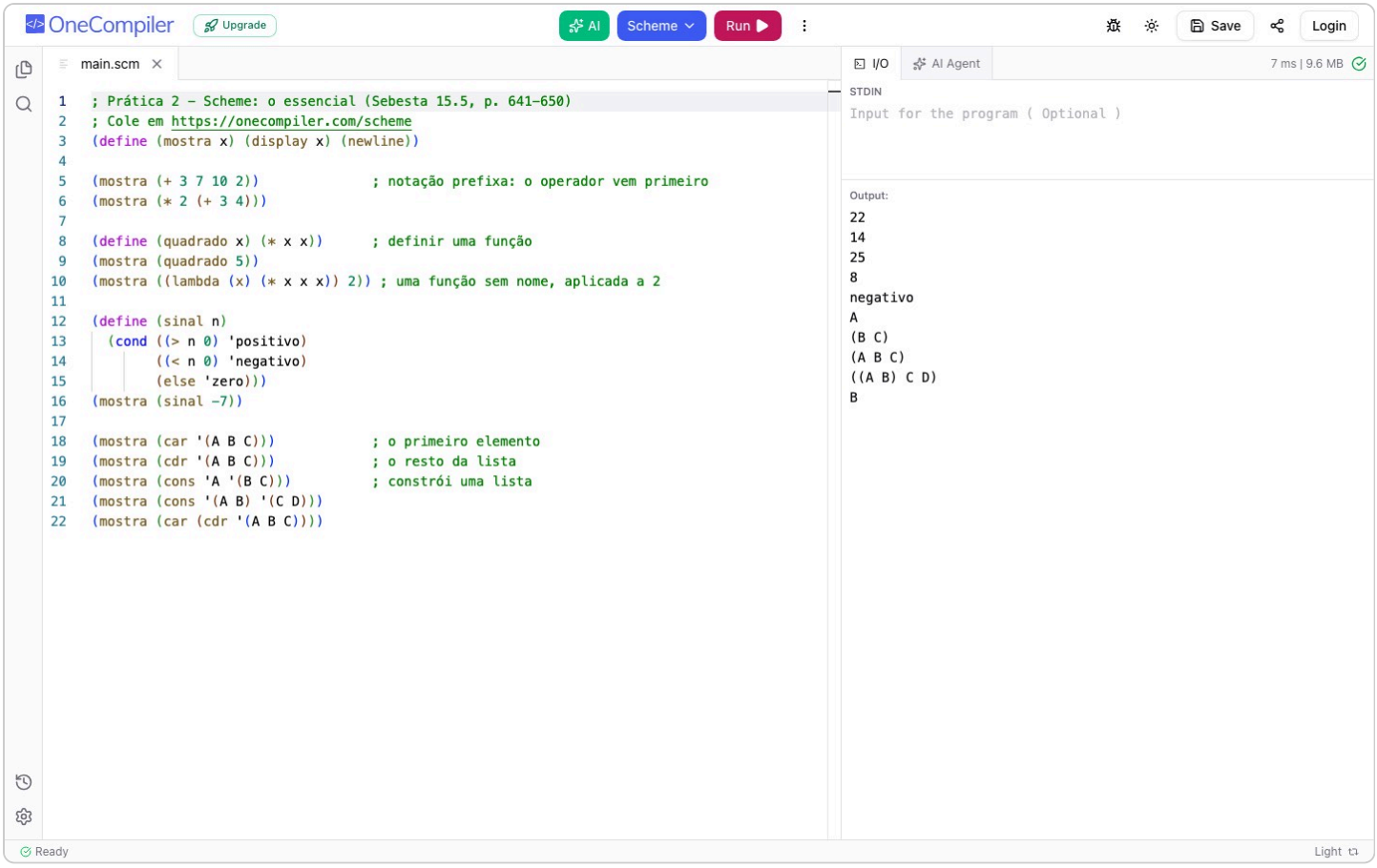
(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

não respondido



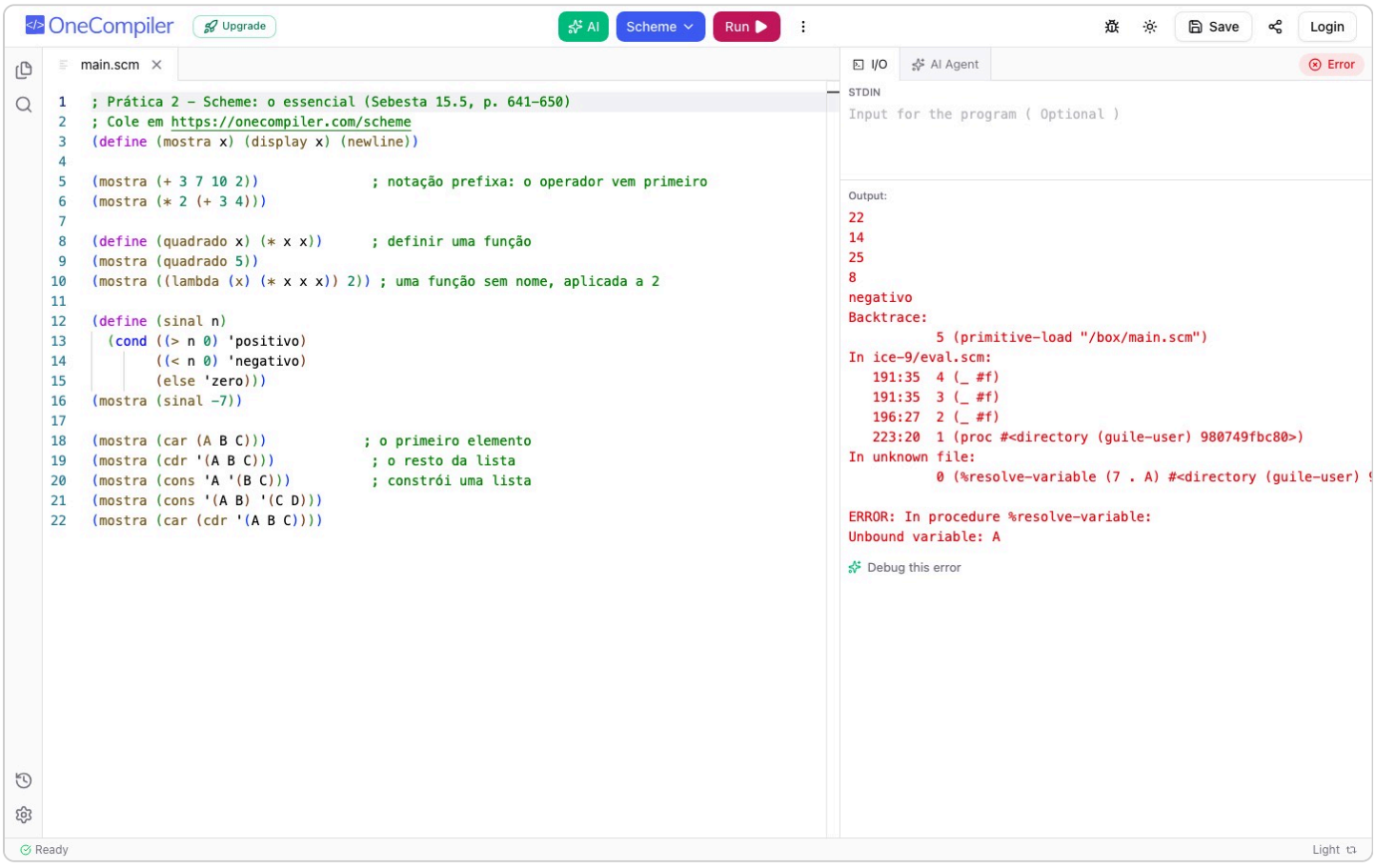

```
; Prática 2 – Scheme: o essencial (Sebesta 15.5, p. 641–650)
; Cole em https://onecompiler.com/scheme
(define (mostra x) (display x) (newline))

(mostra (+ 3 7 10 2))              ; notação prefixa: o operador vem primeiro
(mostra (* 2 (+ 3 4)))

(define (quadrado x) (* x x))      ; definir uma função
(mostra (quadrado 5))
(mostra ((lambda (x) (* x x x)) 2)) ; uma função sem nome, aplicada a 2

(define (sinal n)
  (cond ((> n 0) 'positivo)
        ((< n 0) 'negativo)
        (else 'zero)))
(mostra (sinal -7))

(mostra (car '(A B C)))            ; o primeiro elemento
(mostra (cdr '(A B C)))            ; o resto da lista
(mostra (cons 'A '(B C)))          ; constrói uma lista
(mostra (cons '(A B) '(C D)))
(mostra (car (cdr '(A B C))))
```



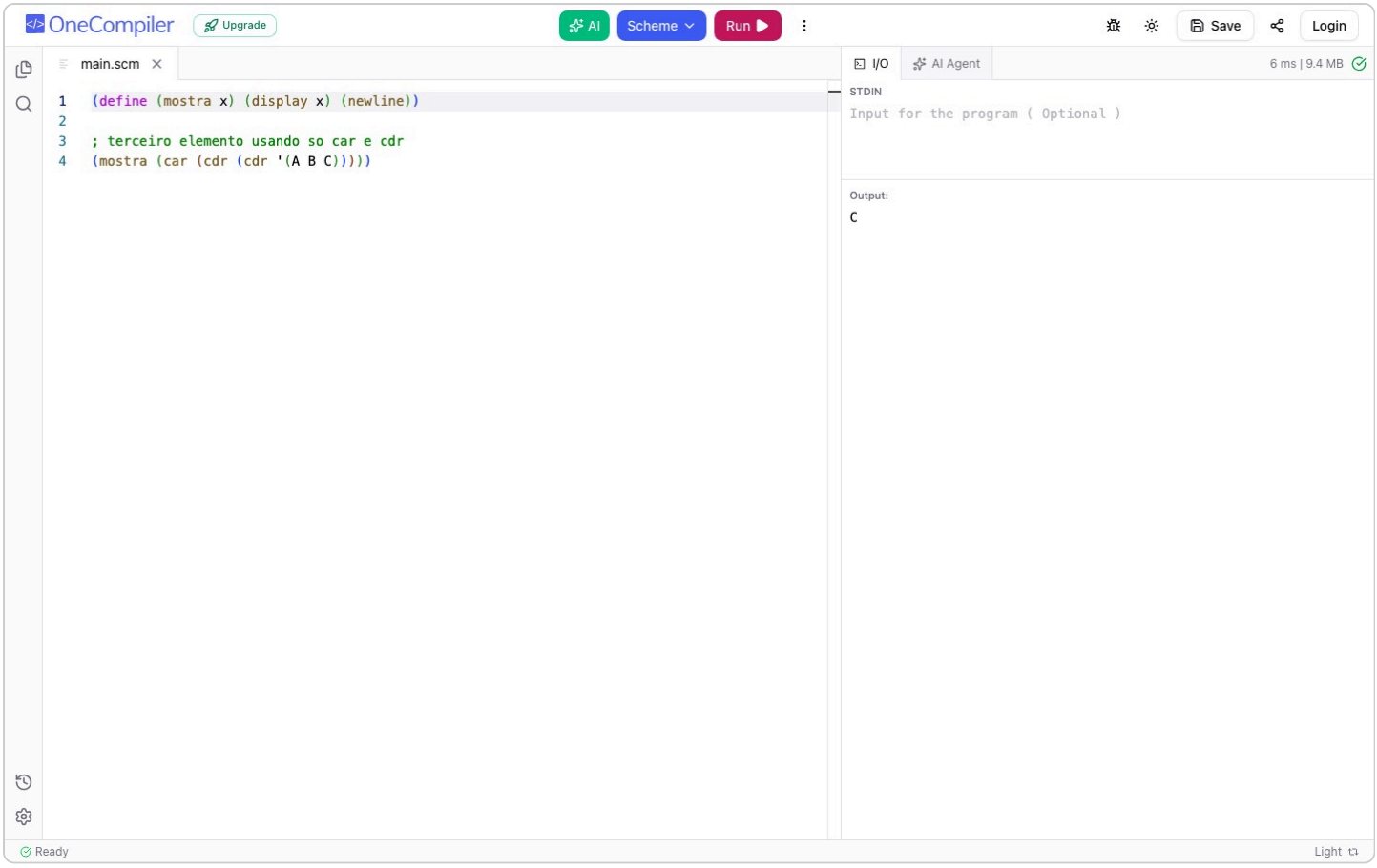

```
; Prática 2 – Scheme: o essencial (Sebesta 15.5, p. 641–650)
; Cole em https://onecompiler.com/scheme
(define (mostra x) (display x) (newline))

(mostra (+ 3 7 10 2))              ; notação prefixa: o operador vem primeiro
(mostra (* 2 (+ 3 4)))

(define (quadrado x) (* x x))      ; definir uma função
(mostra (quadrado 5))
(mostra ((lambda (x) (* x x x)) 2)) ; uma função sem nome, aplicada a 2

(define (sinal n)
  (cond ((> n 0) 'positivo)
        ((< n 0) 'negativo)
        (else 'zero)))
(mostra (sinal -7))

(mostra (car (A B C)))            ; o primeiro elemento
(mostra (cdr '(A B C)))            ; o resto da lista
(mostra (cons 'A '(B C)))          ; constrói uma lista
(mostra (cons '(A B) '(C D)))
(mostra (car (cdr '(A B C))))
```



2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo é a abreviação de QUOTE, ele impede que o Scheme avalie a lista e faz com que ela seja tratada como dado (uma lista literal de símbolos). Sem ele, o Scheme aplicou a regra de avaliação normal: tratou (A B C) como uma chamada de função, com A sendo a função e B e C os parâmetros. Como não existe nenhuma variável/função chamada A, deu o erro "Unbound variable: A".

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C))))

Resultado no OneCompiler: C
(cdr tira o A → (B C); cdr de novo → (C); car → C)
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

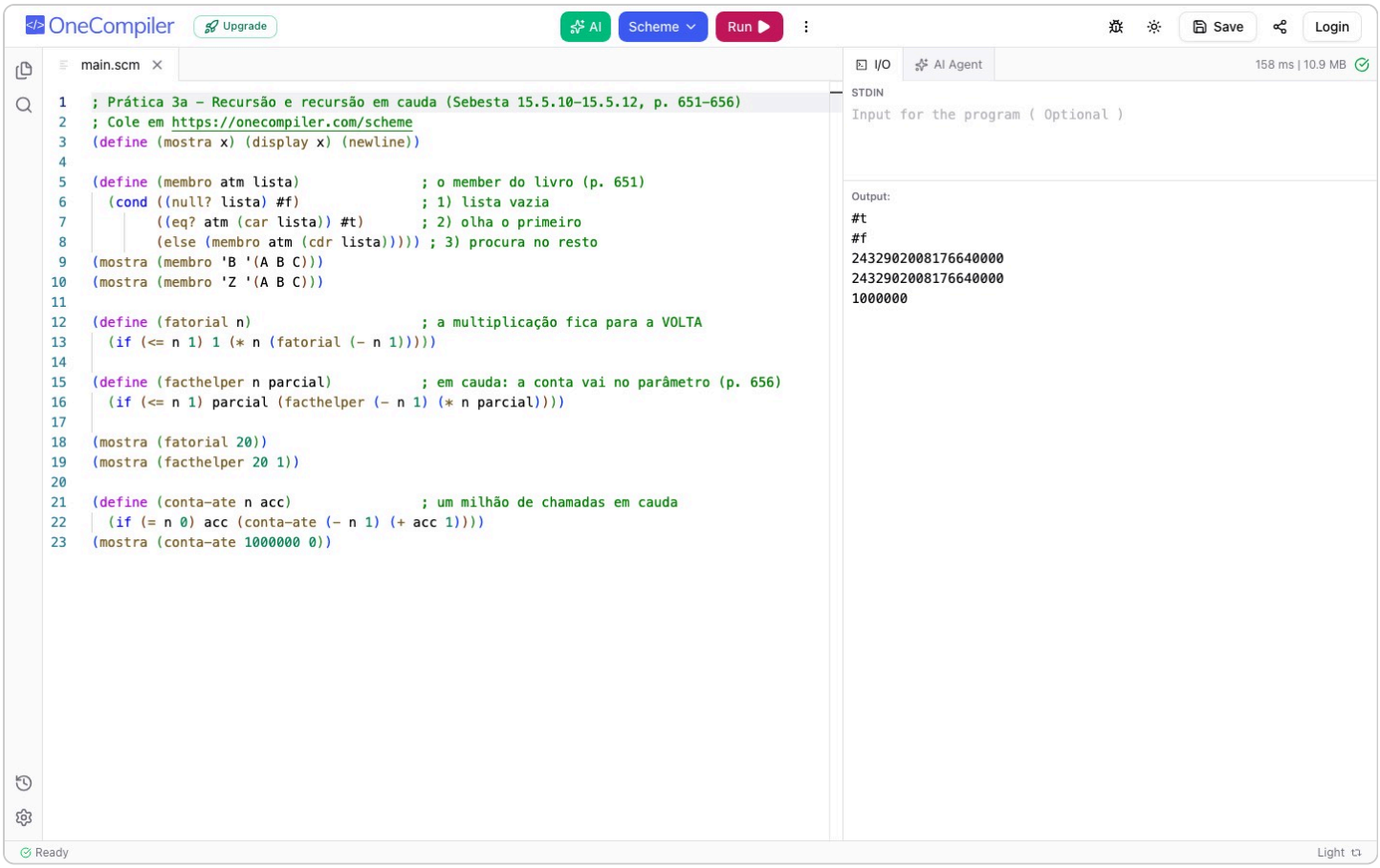
n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).
n = 6: comum em 12 passos, até 5 multiplicações esperando (expressão de até 42 caracteres); em cauda em 8 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Em Scheme acho que funciona, porque a chamada é em cauda e o Scheme otimiza isso. Em Python acho que vai dar erro de recursão, porque o Python tem limite de profundidade de umas 1000 chamadas.

Saída do Scheme

não respondido




```scheme
; Prática 3a – Recursão e recursão em cauda (Sebesta 15.5.10–15.5.12, p. 651–656)
; Cole em https://onecompiler.com/scheme
(define (mostra x) (display x) (newline))

(define (membro atm lista)              ; o member do livro (p. 651)
  (cond ((null? lista) #f)              ; 1) lista vazia
        ((eq? atm (car lista)) #t)      ; 2) olha o primeiro
        (else (membro atm (cdr lista))))) ; 3) procura no resto
(mostra (membro 'B '(A B C)))
(mostra (membro 'Z '(A B C)))

(define (fatorial n)                    ; a multiplicação fica para a VOLTA
  (if (<= n 1) 1 (* n (fatorial (- n 1)))))

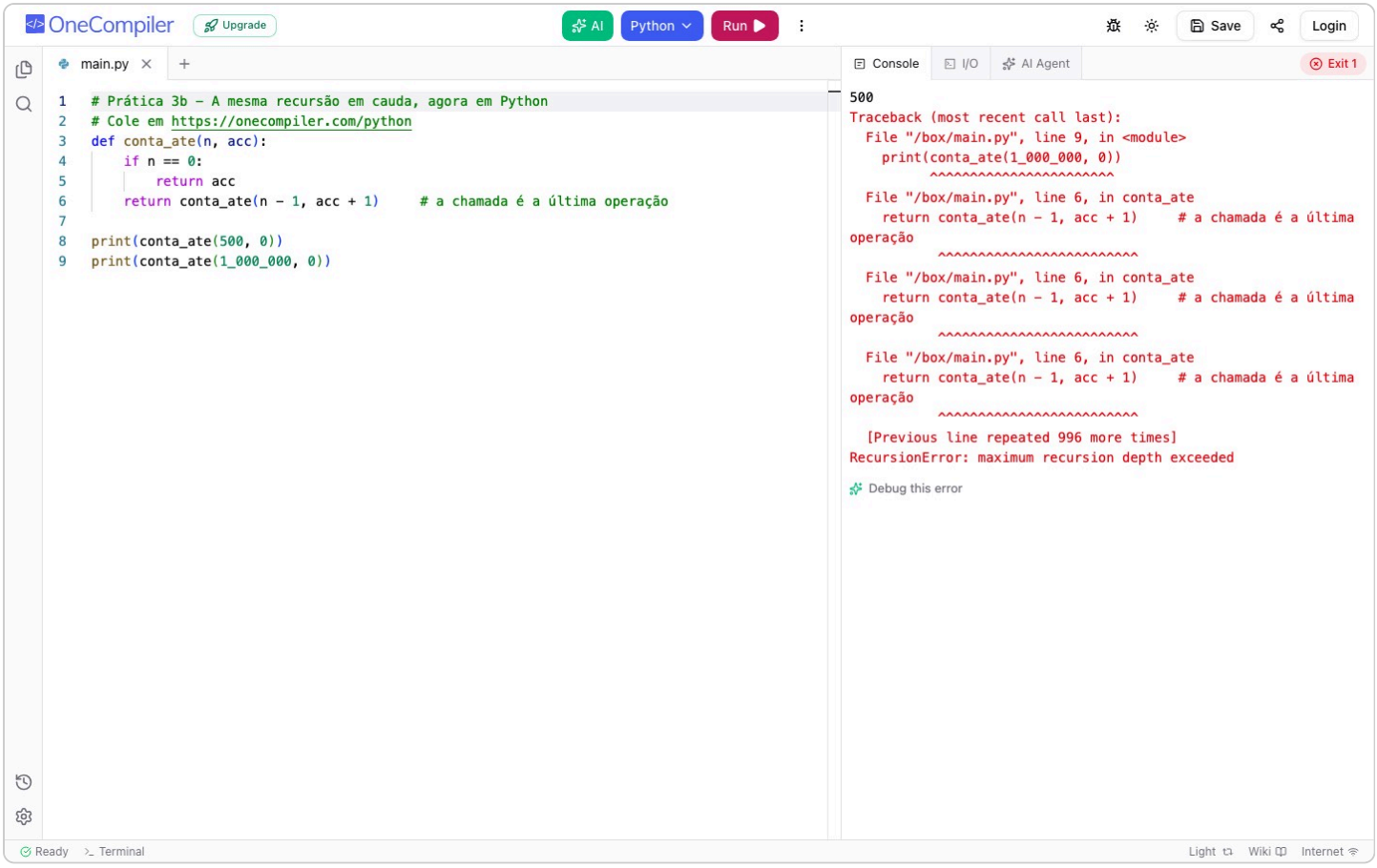
(define (facthelper n parcial)          ; em cauda: a conta vai no parâmetro (p. 656)
  (if (<= n 1) parcial (facthelper (- n 1) (* n parcial))))

(mostra (fatorial 20))
(mostra (facthelper 20 1))

(define (conta-ate n acc)               ; um milhão de chamadas em cauda
  (if (= n 0) acc (conta-ate (- n 1) (+ acc 1))))
(mostra (conta-ate 1000000 0))
```

Saída do Python (as últimas linhas bastam)

```
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
    print(conta_ate(1_000_000, 0))
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```



3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum crescem as multiplicações pendentes, e o tamanho da expressão, com n = 6 chegam a 5 multiplicações esperando e a expressão vai a 42 caracteres. Cada uma corresponde a um registro de ativação que fica na pilha. Na recursão em cauda nada fica pendente e a expressão fica praticamente do mesmo tamanho. Isso acontece porque na comum a multiplicação n * (fatorial (n-1)) só pode ser feita na VOLTA da chamada, então cada chamada precisa esperar. Na em cauda o resultado parcial já vai calculado no parâmetro acumulador e a chamada recursiva é a última operação.

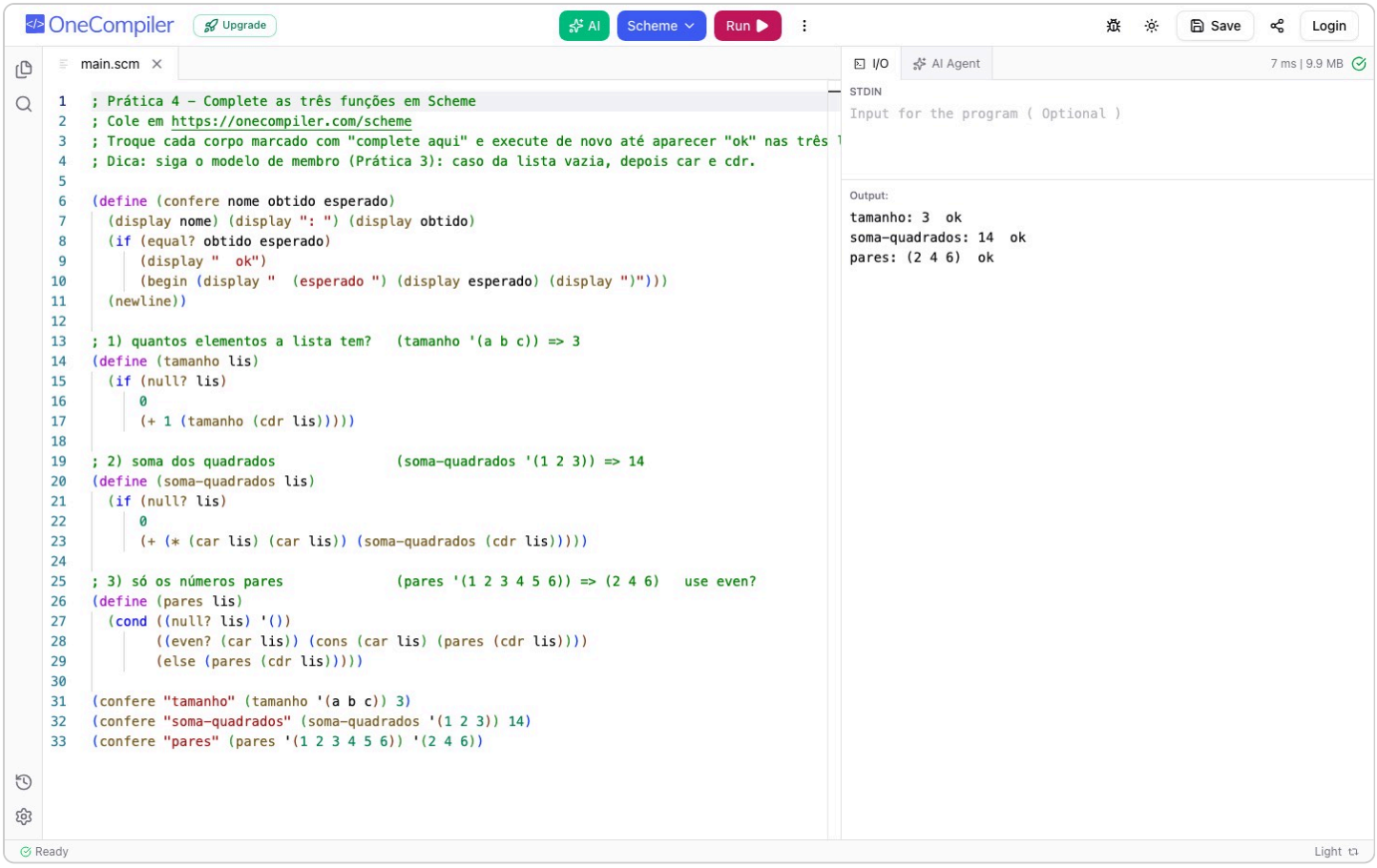
3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme faz a otimização da chamada em cauda: como nada fica esperando, ele reaproveita o mesmo registro de ativação, então a recursão ocupa espaço constante, como um laço, e um milhão de chamadas funciona. O Python NÃO faz essa otimização: cada chamada empilha um novo registro, mesmo sendo em cauda, e ele limita a profundidade da pilha (cerca de 1000 chamadas). Com 500 funcionou, mas com 1.000.000 passou do limite e deu "RecursionError: maximum recursion depth exceeded".

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

não respondido



4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis)) (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis)) (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A mais difícil foi pares, porque tem dois casos além da lista vazia: quando o car é par ele entra com cons na frente do resultado do resto, e quando não é ele é só pulado.
- null?: nas três funções, é o caso base (lista vazia devolve 0, 0 e '()).
- tamanho: não precisa do car, só soma 1 e chama de novo com o cdr.
- soma-quadrados: usa o car para calcular o quadrado (car * car) e soma com a chamada recursiva no cdr.
- pares: usa o car no teste even? e no cons, e o cdr na chamada recursiva dos dois casos.

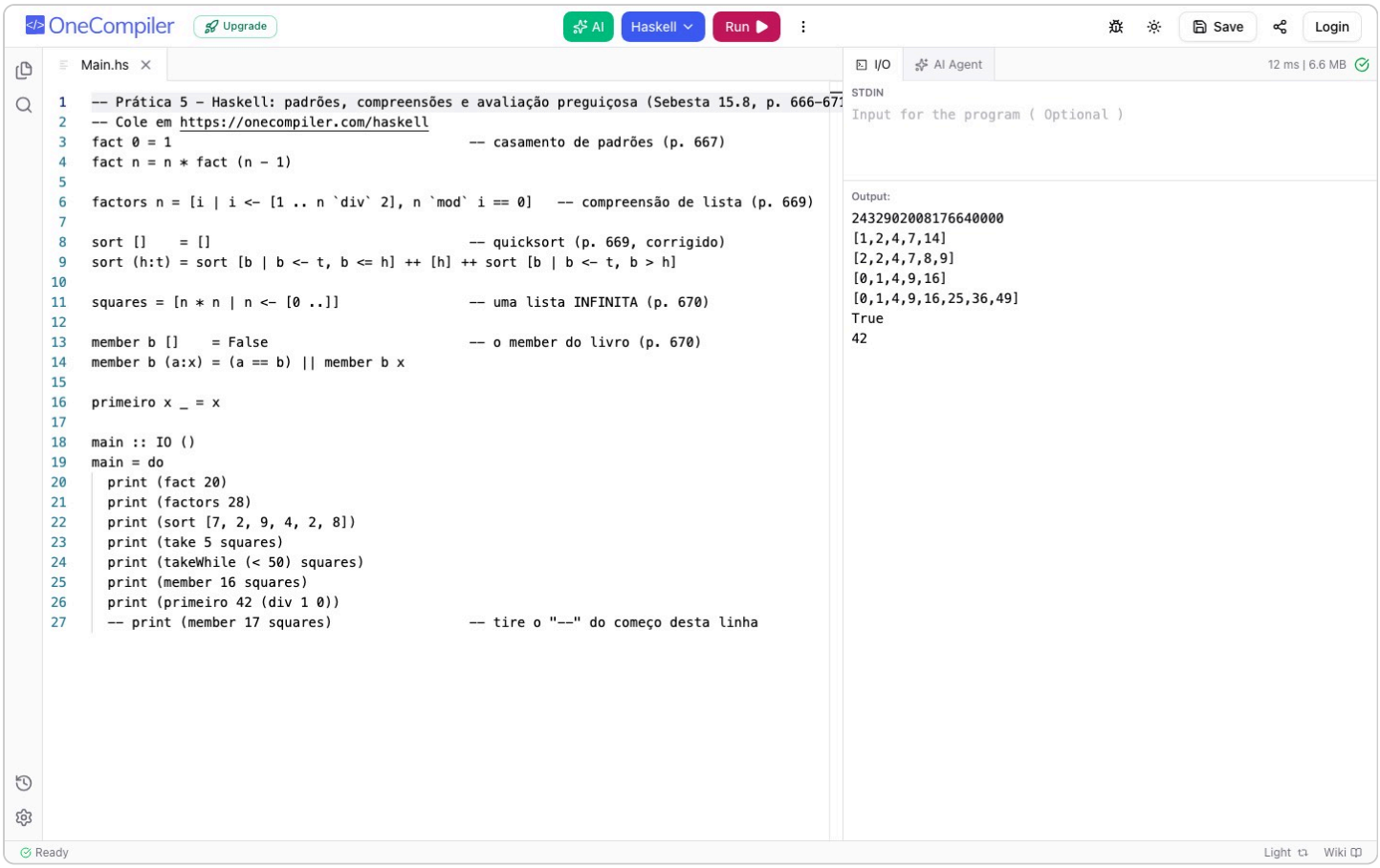
Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares deve imprimir [0,1,4,9,16], porque o Haskell só calcula o que precisa. primeiro 42 (div 1 0) deve imprimir 42 sem erro, porque o segundo parâmetro não é usado e então nunca é calculado.

Saídas (original e com member 17)

não respondido




```haskell
-- Prática 5 - Haskell: padrões, compreensões e avaliação preguiçosa (Sebesta 15.8, p. 666-67
-- Cole em https://onecompiler.com/haskell
fact 0 = 1                                  -- casamento de padrões (p. 667)
fact n = n * fact (n - 1)

factors n = [i | i <- [1 .. n `div` 2], n `mod` i == 0]   -- compreensão de lista (p. 669)

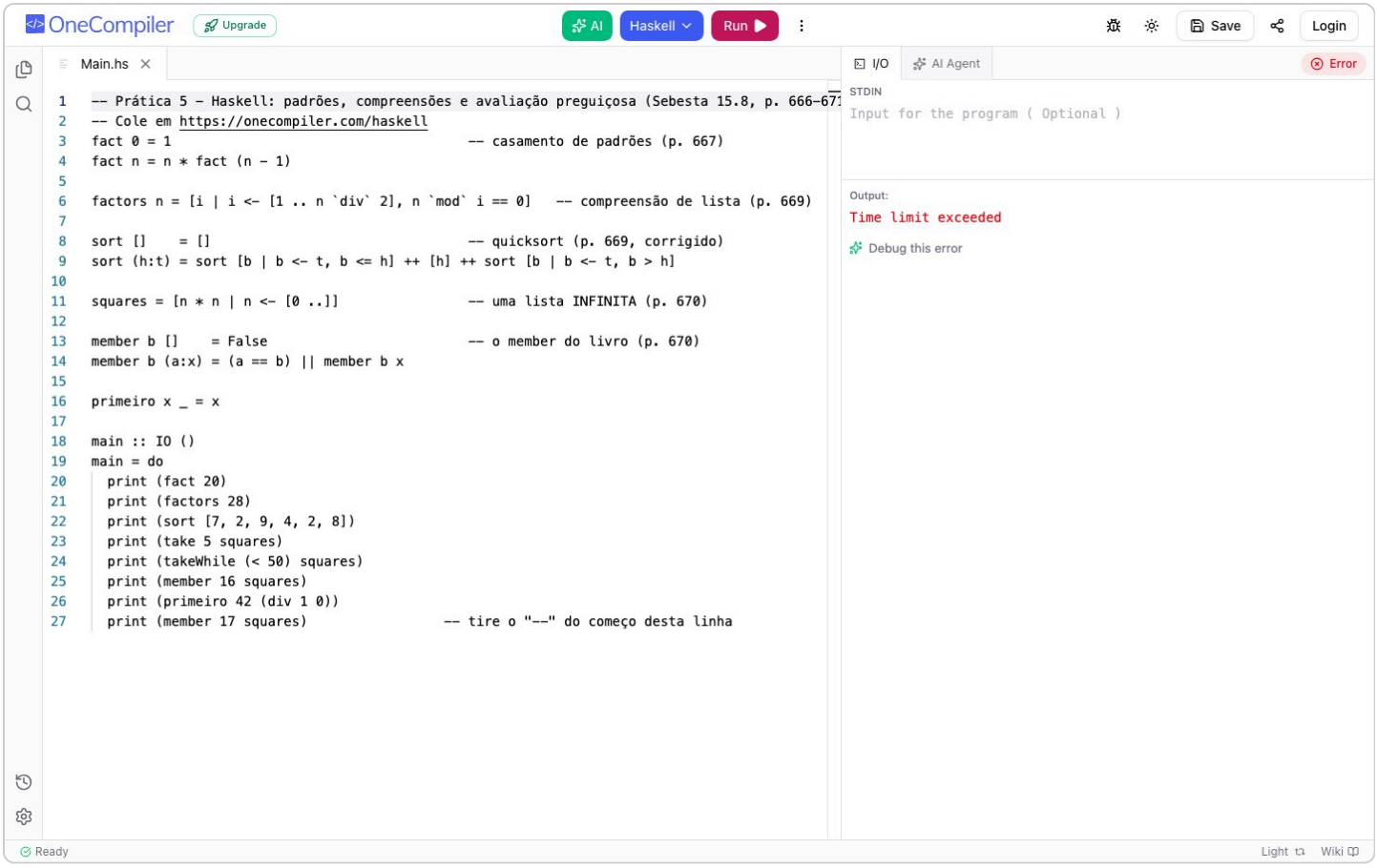
sort []    = []                             -- quicksort (p. 669, corrigido)
sort (h:t) = sort [b | b <- t, b <= h] ++ [h] ++ sort [b | b <- t, b > h]

squares = [n * n | n <- [0 ..]]             -- uma lista INFINITA (p. 670)

member b []    = False                      -- o member do livro (p. 670)
member b (a:x) = (a == b) || member b x

primeiro x _ = x

main :: IO ()
main = do
  print (fact 20)
  print (factors 28)
  print (sort [7, 2, 9, 4, 2, 8])
  print (take 5 squares)
  print (takeWhile (< 50) squares)
  print (member 16 squares)
  print (primeiro 42 (div 1 0))
  -- print (member 17 squares)              -- tire o "--" do começo desta linha
```



5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

Por causa da avaliação preguiçosa: os elementos de squares só são calculados quando alguém precisa deles. take 5 só pede os 5 primeiros, então só esses são gerados e o resto da lista infinita nunca é calculado (o mesmo vale para takeWhile, que para no primeiro >= 50). Já primeiro x _ = x ignora o segundo parâmetro; como o Haskell é não estrito, o parâmetro div 1 0 nunca é avaliado, então a divisão por zero nunca acontece.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

O programa nunca terminou e o OneCompiler cortou com "Time limit exceeded". member percorre squares procurando o 17; como 17 não é quadrado perfeito ele nunca é encontrado, e como a lista é infinita ela nunca chega no caso [] que devolveria False. Com 16 funcionou porque 16 aparece na 5ª posição, o (==) dá True e o || para a busca ali.

Correção: como squares é crescente, dá para desistir assim que o elemento passar do valor procurado (o member2 da p. 670):

```
member2 n (m:x)
 | m < n    = member2 n x
 | m == n   = True
 | otherwise = False
```

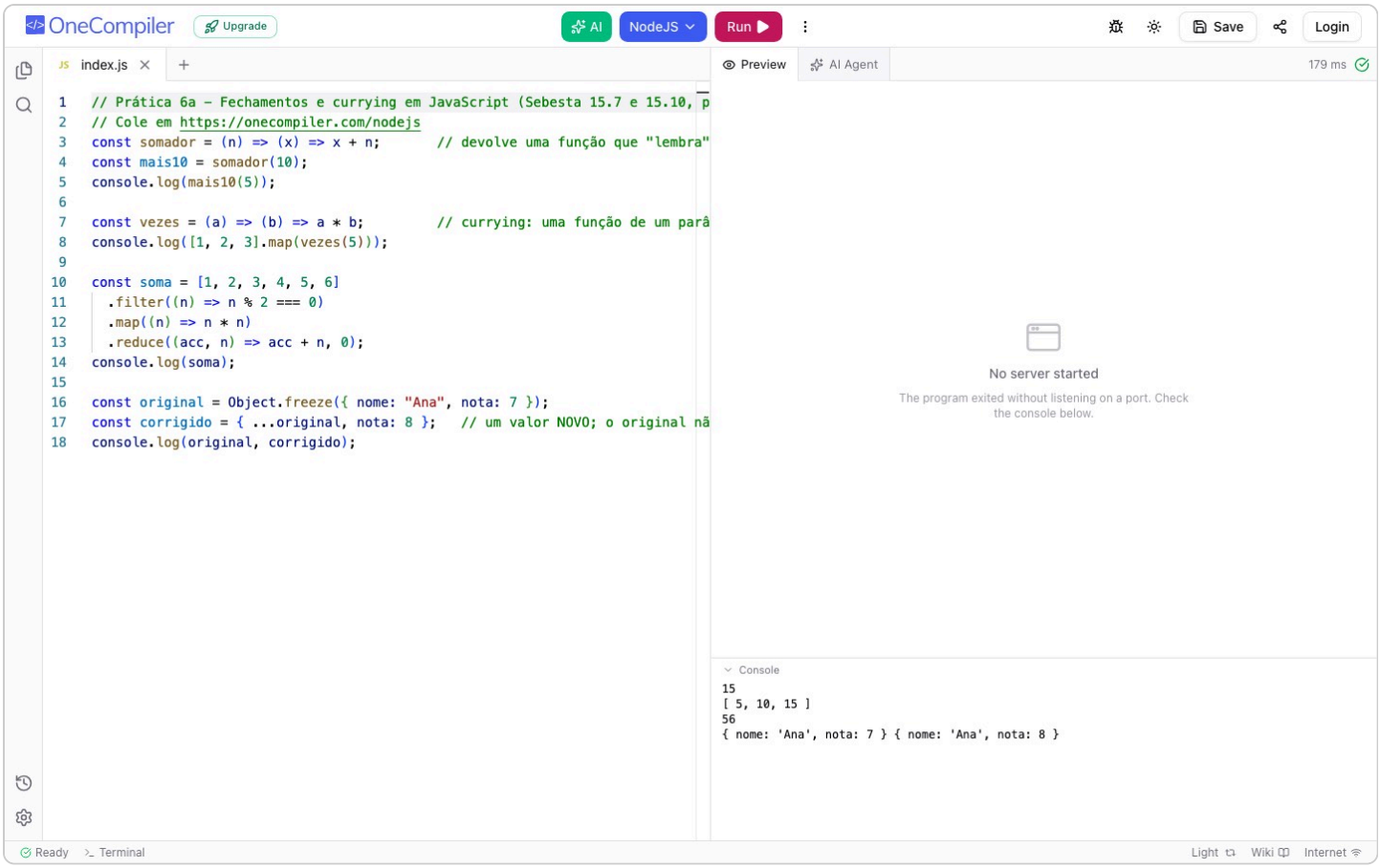
Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) imprime 15 e [1, 2, 3].map(vezes(5)) imprime [5, 10, 15].

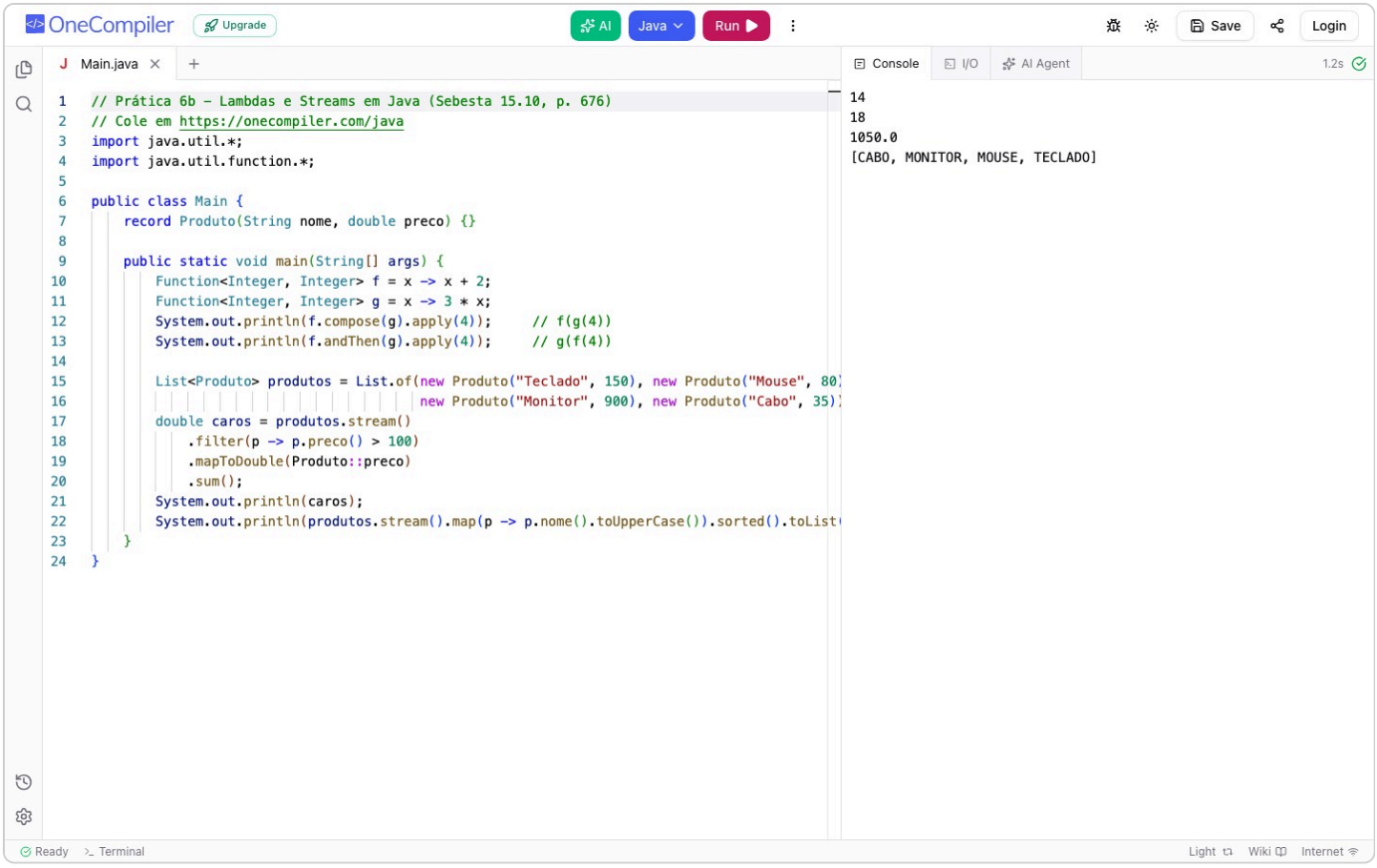
Saída do JavaScript

não respondido



Saída do Java

não respondido



6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

Porque a função devolvida é um fechamento (closure): quando ela é criada, guarda uma ligação com o ambiente onde foi definida, que tem n = 10. Essa variável capturada continua existindo enquanto a função mais10 existir, mesmo depois que somador(10) terminou. Então, ao chamar mais10(5), ela usa o n = 10 guardado e calcula 5 + 10 = 15.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Porque a ordem de aplicação é diferente. f.compose(g) aplica primeiro g e depois f: f(g(4)) = f(12) = 14. f.andThen(g) aplica primeiro f e depois g: g(f(4)) = g(6) = 18. O que corresponde ao compor(f, g) da Prática 1 é o f.compose(g), que também deu 14.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
/// Java
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);
        int soma = nums.stream()
            .filter(n -> n % 2 == 0)
            .mapToInt(n -> n * n)
            .sum();
        System.out.println(soma);
```

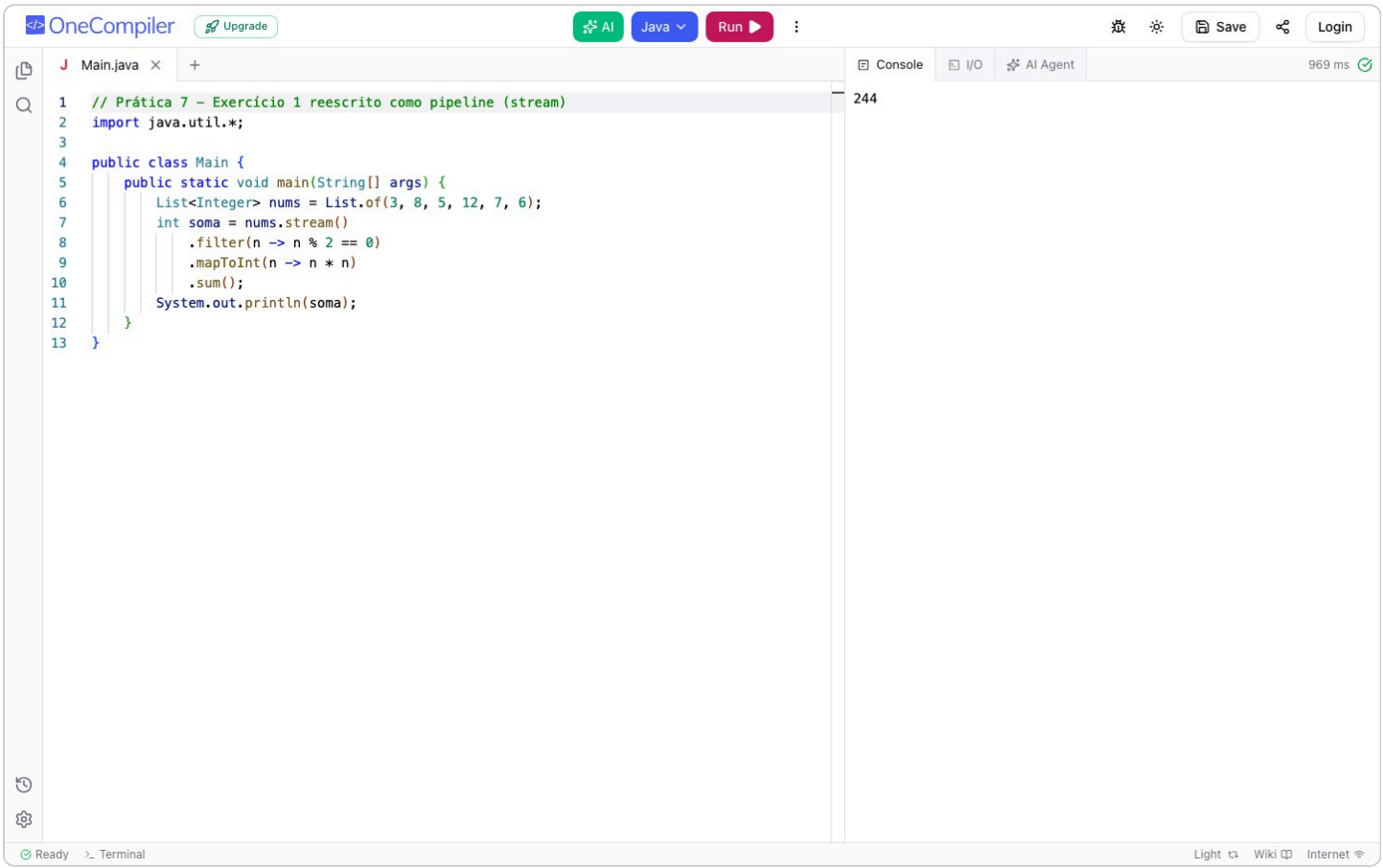
```
    }
}
```

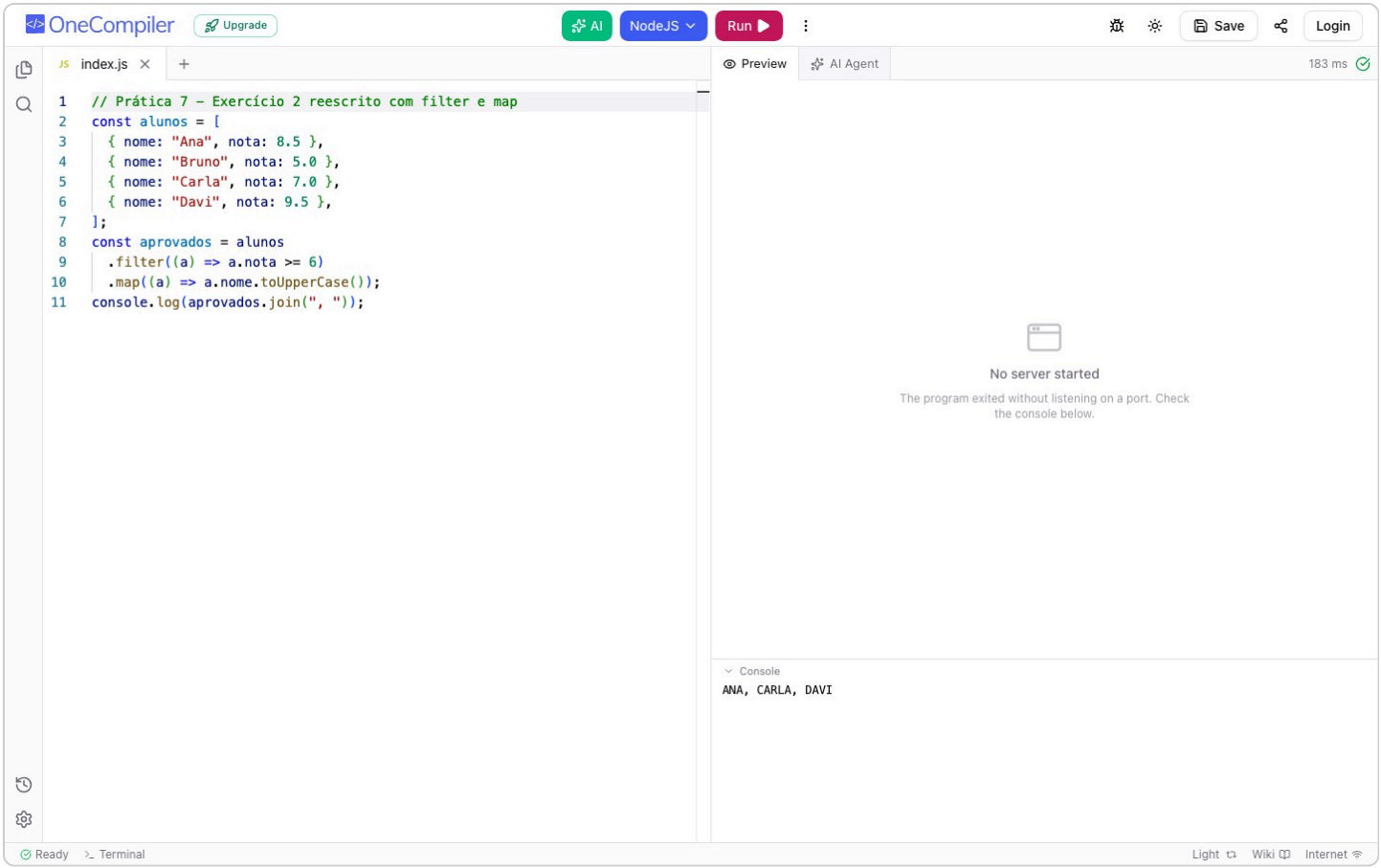
7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
// JavaScript
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];
const aprovados = alunos
  .filter((a) ⇒ a.nota ⩾ 6)
  .map((a) ⇒ a.nome.toUpperCase());
console.log(aprovados.join(", "));
```

Saídas das suas versões

não respondido





7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

Para ler, a funcional é mais fácil: ela descreve o que é feito (filtra os pares, eleva ao quadrado, soma) em vez de como (índice, if, variável acumuladora), e não tem estado mudando. Para depurar, a versão com laço é mais fácil: dá para colocar um print ou breakpoint dentro do laço e ver o valor de cada variável a cada volta. No pipeline é preciso quebrar a cadeia em variáveis intermediárias ou usar algo como peek() no Java / um map com console.log no JavaScript.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em Haskell é mais fácil de entender: é uma linha que diz exatamente o que se calcula (gera 1..n, eleva ao cubo, soma), sem variável que muda, índice e condição de parada para acompanhar. A versão em C obriga a simular o laço na cabeça.
Eu aceitaria o custo de ~2x em sistemas onde corretude, legibilidade e manutenção pesam mais que velocidade bruta: sistemas web e de negócio, processamento e transformação de dados, ferramentas internas e sistemas concorrentes (sem estado compartilhado fica mais fácil paralelizar). Não aceitaria em sistemas onde desempenho e uso de memória são críticos: embarcados com pouco recurso, tempo real (controle industrial, automotivo), drivers, motores de jogos e computação científica de alto desempenho.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi o Haskell conseguir trabalhar com uma lista infinita e ignorar uma divisão por zero só porque nunca precisou calcular aquilo, enquanto o Python estoura a pilha numa recursão em cauda que o Scheme roda um milhão de vezes sem problema.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro está no primeiro filtro: está impresso "b <- h" quando deveria ser "b <= h". Com <-, o Haskell entende que b é tirado da lista h, então infere que h é uma lista. Por isso o compilador inferiu o tipo sort :: [a] -> [[a]] (com h :: [a] e t :: [[a]]), e no segundo filtro, b > h, ele não consegue casar os tipos: "Couldn't match expected type 'a' with actual type '[a]'".
Correção:

```
sort (h:t) = sort [b | b <- t, b <= h]
        ++ [h] ++
        sort [b | b <- t, b > h]
```